

BACKEND ENGINEER · JEON MINGYU

빠르게 선점하고, 성공은 늦게 선언합니다

Redis에서 자리를 잡더라도 MySQL에 실제 쿠폰이 만들어진 뒤에만 성공을 반환했습니다. 트래픽이 몰릴수록 성공의 기준을 더 엄격하게 잡았습니다.

Java 21

Spring Boot

MySQL

Redis Lua

Kafka

20,000

신청 시도 · 60초 Ramp

0

전송 실패 · 예상 밖 응답

951.19ms

쿠폰 신청 응답 p95

NAME

전민규

ROLE

Backend Engineer

GITHUB

github.com/minggyure

FEATURED PROJECT · CLUTCH

제가 맡은 경계는 '신청'부터 '쿠폰함'까지였습니다

PROJECT

CLUTCH

LoL 경기 사건이 발생하면 제한 수량의 쿠폰을 단계별 선착순으로 발급하는 서비스입니다.

6인 팀 · 2026.08.11 — 08.31

담당 · 쿠폰 발급 백엔드

결과 · LG U+ URECA 종합 프로젝트 우수상

✓ 쿠폰 신청 API와 상태 전이

✓ Redis Lua 재고·중복 제어

✓ MySQL 실제 쿠폰 생성

✓ 저장 실패 보상 처리

✓ Redis 장애 복구

✓ SSE 실시간 잔여 재고

✓ Outbox·Kafka 결과 전달

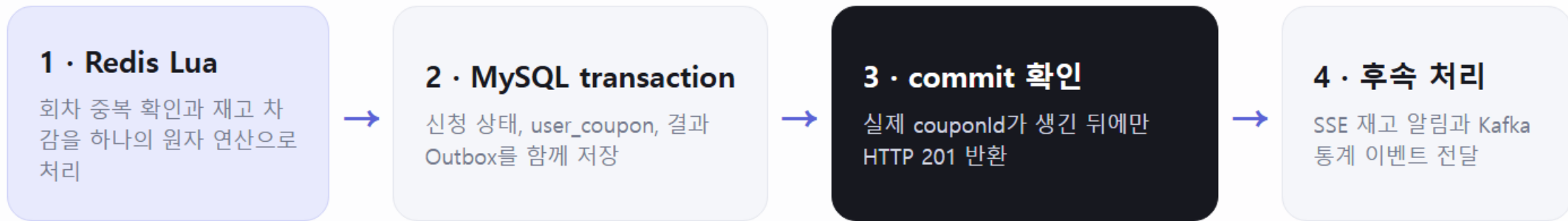
✓ 관리자 통계 조회 개선

팀 기능과 제 기여를 섞지 않았습니다. 경기 수집·배팅·시청 포인트는 다른 팀원이 맡았고, 저는 쿠폰 발급 흐름과 그 후속 통계를 구현했습니다.

DESIGN DECISION 01

Redis는 자리를 잡고, MySQL이 발급을 확정합니다

재고 차감만 성공한 상태를 사용자에게 '발급 완료'로 보여주면 쿠폰함과 응답이 어긋납니다.



중복·초과 발급 방지

Lua Script가 재고와 사용자 집합을 같이 바꿉니다.

저장 실패 보상

DB 저장이 실패하면 Redis 재고와 당첨 기록을 되돌립니다.

성공의 단일 기준

최종 발급 사실은 MySQL의 user_coupon으로 정의합니다.

DESIGN DECISION 02

Redis가 죽으면, 발급도 잠시 멈춥니다

재고 상태를 모르는 상황에서 MySQL로 우회하면 두 저장소가 각자 재고를 판단합니다. 그래서 **503으로 안전하게 실패**시키고, 애플리케이션이 MySQL을 기준으로 복구합니다.

1 UNAVAILABLE

연결 장애나 필수 키 누락을 감지하면
새 발급을 503으로 차단

2 RECOVERING

실제 user_coupon 수와 당첨 사용자
집합을 MySQL에서 조회

3 REBUILD

남은 재고·당첨자·회차 컨텍스트를
Redis에 원자적으로 재구축

4 READY

재구축 값이 기대값과 같을 때만 발급
을 다시 허용

FAULT INJECTION · STOCK 100

20장 발급 → Redis 중단 → 503 → 복구 →
80장 발급

100

최종 실제 쿠폰

0

초과·중복 발급

101번째 요청은 정상 품질.

복구의 기준을 Redis가 아닌 영속 데이터에 두었습니다.

TROUBLESHOOTING 01

공통 success_count 한 행이 모든 요청을 세웠습니다

Redis는 빨랐지만, 성공한 5,000개 transaction이 같은 행의 X-lock 앞에서 다시 직렬화됐습니다.

BEFORE · HOT ROW

락을 바꿔도 대기열은 남았습니다

5,000 Redis 성공 → 같은 행 UPDATE → HTTP timeout

낙관적 락은 재시도를 늘리고, 비관적 락은 대기를 명시할 뿐이었습니다.

19/20

Hikari active

178

connection 대기

2,914

행 잠금 / 분

1,115s

잠금 대기 / 분



AFTER · SEPARATE

발급과 조회용 집계를 분리했습니다

user_coupon 저장 → 5초 뒤 GROUP BY → success_count 보정

발급 transaction에는 실제 쿠폰 저장만 남기고, 조회용 성공 수량은 최대 5초 지연을 허용했습니다.

추가 방어 · 전용 인덱스 + 단일 GROUP BY + MySQL named lock으로 다중 인스턴스 중복 집계를 차단했습니다.

TROUBLESHOOTING 02

6,216건은 서버 오류가 아니었습니다

DB pool을 늘려도 dial: i/o timeout이 남았습니다. 요청이 애플리케이션에 닿기 전 경로를 분리해 확인했습니다.

Docker k6

6,216 failures

k6 container → WSL / NAT → Windows TCP → Tailscale

백엔드·재고·사용자 수는 정상인데 HTTP 상태 코드 자체를 받지 못했습니다. 이 실행은 정합성 성공으로 판정하지 않았습니다.

Windows native k6

0 failures

k6.exe → Windows TCP → Tailscale → Backend

부하 발생기의 Docker NAT 구간만 제거하고 나머지 조건은 유지했습니다.

10,000

발급 성공

10,000

정상 품질

0

전송·예상 밖 실패

951.19ms

Claim p95 · 이전 1.84s

측정 조건 · 최대 20,000 VU에 도달하는 60초 Ramp입니다. 요청 20,000건을 같은 순간에 발사한 Burst 결과는 아닙니다.

PROOF AT SCALE

351만 요청 이력을 수정하지 않고 검사했습니다

운영성 데이터를 고치지 않는 읽기 전용 일관 스냅샷에서 참조·상태·중복·재고를 전체 집계했습니다.

1,000,003

사용자

3,517,209

발급 요청

797,209

실제 쿠폰

REPEATABLE READ + READ ONLY

건수·위반 수·CRC32 fingerprint만 기록하고 개인정보는 출력하지 않았습니다.

FAIL CHECKS · ALL ZERO

0 고아 참조

0 성공 요청의 쿠폰 누락

0 사용자·회차 중복 요청

0 중복 쿠폰

0 수량 초과 발급

0 상태·시각 불일치

0 장기 PENDING

0 만료 후 OPEN 회차

검사에서 발견한 **WARN**도 숨기지 않았습니다. DB에는 있었지만 Java enum에 없던 CANCELLED·EXPIRED 계약을 맞추고, 집계 인덱스 누락과 항목별 COUNT를 후속 PR에서 정리했습니다.

DESIGN DECISION 03

Kafka는 발급이 아니라, 반복 조회를 줄이기 위해 썼습니다

사용자 응답은 MySQL commit으로 끝납니다. Kafka는 확정된 결과를 관리자 일별 통계에 늦게 반영해도 되는 후속 경로입니다.

BEFORE · 요청마다 집계

최근 7일 원본을 매번 GROUP BY

634,917**AFTER · 미리 누적**

Kafka Consumer가 KST 일별 통계를 먹등하게 증가

7 rows

QUERY SEGMENT

447ms → 0.0296ms

같은 로컬 MySQL 세션에서 원본 집계와 일별 7행 조회를 비교했습니다.

왜 Kafka였나

발급과 통계의 장애·처리 시간을 분리하고, Outbox 재발행과 messageId 먹등성으로 중복 누적을 막기 위해서였습니다.

한계도 구분했습니다. 0.0296ms는 쿼리 구간입니다. 원격 관리자 API 전체가 같은 비율로 빨라졌다는 뜻은 아닙니다.

DAENGGO · COMMUNITY BACKEND

게시글·댓글을 실제 인증 흐름에 연결했습니다

반려견 동반 장소·산책·커뮤니티를 연결한 4인 팀 프로젝트에서 커뮤니티 API와 팀 실행 환경을 맡았습니다.

게시글·댓글 CRUD

Entity, DTO, Repository부터 등록·조회·수정·삭제 API까지 구현

이미지·조회수

게시글 이미지 등록·편집과 상세 조회 시 조회수 반영

작성자 권한

JWT 사용자와 데이터 소유자를 비교해 수정·삭제 제한

팀 실행 환경

Docker Compose와 Nginx로 FE·BE·MySQL 실행 절차 통합

통합 오류 수정

임시 사용자 로직을 제거하고 인증 결과를 권한 검증에 사용

기여 근거

PR #8 · #12 · #15 · #18 · #23 · #30 · #33

UPDATE / DELETE FLOW

JWT에서 로그인 사용자 확인



게시글·댓글 작성자 조회



동일 사용자면 변경 · 아니면 거절

Java 21

Spring Boot

JPA

MySQL

Docker

BACKEND ENGINEER · JEON MINGYU

빠른 코드보다, 끝까지 설명할 수 있는 결과를 남깁니다

01 · BOUNDARY

성공의 기준을 먼저 정합니다

Redis 선점과 MySQL 최종 발급을 분리했습니다.

02 · FAILURE

장애를 숨기지 않습니다

503으로 멈추고, 영속 데이터를 기준으로 복구했습니다.

03 · EVIDENCE

조건과 한계를 같이 적습니다

558개 테스트와 부하·정합성 결과를 문서로 남겼습니다.

GITHUB

github.com/minggure

CLUTCH CONTRIBUTIONS

작성한 Pull Request 보기

DAENGGO CONTRIBUTIONS

작성한 Pull Request 보기